

Accepted Manuscript

Title: Improving variable neighborhood search to solve the traveling salesman problem

Author: Samrat Hore Aditya Chatterjee Anup Dewanji

PII: S1568-4946(18)30179-0

DOI: <https://doi.org/doi:10.1016/j.asoc.2018.03.048>

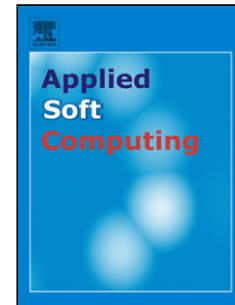
Reference: ASOC 4797

To appear in: *Applied Soft Computing*

Received date: 22-9-2017

Revised date: 23-2-2018

Accepted date: 28-3-2018



Please cite this article as: Samrat Hore, Aditya Chatterjee, Anup Dewanji, Improving variable neighborhood search to solve the traveling salesman problem, *Applied Soft Computing Journal* (2018), <https://doi.org/10.1016/j.asoc.2018.03.048>

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting proof before it is published in its final form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.

Improving variable neighborhood search to solve the traveling salesman problem

Samrat Hore, Department of Statistics, Tripura University

Aditya Chatterjee, Department of Statistics, University of Calcutta

Anup Dewanji, Applied Statistics Unit, Indian Statistical Institute, Kolkata

Abstract

The Traveling Salesman Problem (*TSP*) is one of the classical combinatorial optimization problems and has wide application in various fields of science and technology. In the present paper, we propose a new algorithm for solving the *TSP* that uses the variable neighborhood search (*VNS*) algorithm coupled with a stochastic approach for finding the optimal solution. Such neighborhood search with various other local search algorithms, named as *VNS* – 1 and *VNS* – 2, has been reported in the literature. The proposed algorithm is compared in detail with these algorithms, in the light of two benchmark *TSP* problems (one being symmetric while the other is asymmetric) suggested in the *TSPLIB* dataset in programming language R, along with two asymmetric problems obtained through simulation experiment. The present algorithm has been found to perform better than the conventional algorithms implemented in R for solving *TSP*'s, and also, on an average, found to be more effective than the *VNS* – 1 and the *VNS* – 2 algorithms. The performance of the proposed algorithm has also been tested on 60 benchmark symmetric *TSP*s from the *TSPLIB* dataset. Apart from solving the *TSP*, the flexibility of the proposed hybrid algorithm to solve some other optimization problems related to other disciplines has also been discussed.

Keywords: Neighborhood structure, Hybrid *VNS*, Stopping rule, Near-optimal solution, Cost matrix.

1. Introduction

The *TSP* is a well-known, popular, and an extensively studied problem in the field of combinatorial optimization. It has immense applications in the field of engineering science like designing hardware devices and radio electronic devices in communications, in the architecture of computational networks, designing of microchips, DNA sequencing [1], data

association, vehicle routing [2, 3], data transmission in computer networks [4], image processing and pattern recognition [5], to quote a few. The concept of the *TSP* is very simple but yet it remains one of the most challenging problems in operations research. More succinctly, it is an optimization problem to find the shortest connectivity among all the nodes in a closed network by touching all the nodes exactly once. Although seems simple, the computer running time for an extensive search approach lies within a polynomial factor of $O(n!)$, where n is the number of nodes, thus making the problem most difficult, even for $n = 20$. In theoretical computer science, such kind of problem is known as the classical *NP*-complete problem [6], which means, for the worst-case, the running time for any algorithm for the *TSP* increases super-polynomially (or, perhaps exponentially) with the number of nodes [7]. The graph-theoretic method identifies the problem as finding the Hamiltonian cycle with the least weight for a given complete weighted graph [8].

Since 1950's, researchers are trying to develop various computer intensive algorithms to reach an optimal or near-optimal solution. Among these algorithms, branch and bound [9] has been adopted for solving the *TSP* with smaller scale. Apart from *TSP* with one salesman, one may consider the problem with multiple salesmen, departing from several starting cities and returning back to their respective starting cities. Such *TSP* is known as fixed destination multi-depot multiple traveling salesmen problem (*MmTSP*). Dynamic programming is one of the conventional methods for solving the *TSP*, both symmetric and asymmetric and also *MmTSP*, which starts with a minimum sub-tour and the optimal tour is reached by augmenting the city not yet in the sub-tour [10, 11]. Particle swarm optimization [12, 13] has been invoked to solve the *TSP* with a larger scale say, for $n = 51$ to 76 . Such solution has been successfully applied for vehicle routing problems [2, 14, 15]. Another metaheuristic, known as Ant Colony Optimization has been applied to solve such *MmTSP* [16, 17]. This algorithm mimics the behavior of the real ants in search of food, where every ant leaves an amount of pheromone on the path it passes and chooses the path with more pheromone left on it from the previous ants. Since more ants pass the shorter roots, gradually, more and more ants choose the shorter root and as a result, the colony finds the best way to get to the food. Genetic algorithms are an alternative metaheuristic optimization algorithm which encode a potential solution to a specific problem on a simple chromosome-like data structure, and apply recombination operators to these structures in such a way as to preserve critical information [18–20]. Simulated annealing [21]

22] is one of the widely used algorithms to achieve a global optimal solution for computation intensive combinatorial optimization problems in recent days. Simulated annealing is a method for improving local optimization, and it needs less memory space. However, to find a better solution, simulated annealing algorithm generally requires longer computational time compared to other heuristics of discrete optimization, a brief review of which is discussed in the next few paragraphs.

The simplest of its kind is the Nearest Neighbor Algorithm (*NNA*) [23]. This algorithm starts with a sub-tour, or path, containing a subset of randomly chosen cities from the list of n cities and then adds the nearest city that is yet to visit. The algorithm stops when all cities are included in the tour. An extension to this algorithm is to repeat it with each city as the starting point and then return the best tour found, which is called the Repetitive Nearest Neighbor Algorithm (*RNNA*) [23]. Another approach is known as Insertion Algorithm (*IA*) [23], where it starts with a sub-tour consisting of two arbitrary cities (say, i and j) and then chooses in each step a city k not yet in the sub-tour. This city is inserted between the two cities i and j , such that the insertion cost (i.e., the net increase in the cost of the tour) given by $d(i, k) + d(k, j) - d(i, j)$ is minimum. This insertion process is repeated with the current sub-tour $i \rightarrow k \rightarrow j$ and a fourth city is inserted between either i and k , or k and j , with the objective of minimizing the net increase in the cost of the next sub-tour. As before, the algorithm stops when all the cities are included in the tour. According to Hahsler and Hornik [24], four variants of *IA* are available, depending upon the way the city is to be inserted next is chosen, which are (1) Nearest Insertion (*NI*), (2) Farthest Insertion (*FI*), (3) Cheapest Insertion (*CI*) and (4) Arbitrary Insertion (*AI*), among which the first two are most popular. For the comparative analysis of the proposed algorithm, we have considered only the first two.

Some algorithms based on local improvements of the existing tour through simple tour modifications are also available in the literature. Such algorithms are specified in terms of a class of operations (exchanges or moves) that can be used to convert one tour into another by reducing the tour length until a tour is reached for which no such operation yields an improvement (i.e., a locally optimal tour). Among such simple local search algorithms, the most famous are $2 - Opt$ and $3 - Opt$ algorithms. The $2 - Opt$ algorithm was developed by Croes [25], although it was earlier proposed by Flood [26]. The procedure deletes two edges of the tour, thus breaking it into two paths, and then reconnects those paths in the other

possible way. A modified $2-Opt$ algorithm named as the *Quicker 2-Opt* was introduced in [27], by reducing search space for a selected pair of links, instead of deleting and reconnecting all possible pairs of links in the $2-Opt$ algorithm. In the $3-Opt$ algorithm [28], the exchange replaces up to three edges of the current tour. The idea of $2-Opt$ and $3-Opt$ algorithms may be extended to $k-Opt$ algorithms where the exchange replaces up to k edges of the current tour. This extension by Lin and Kernighan [29] does not use a fixed value for k for its $k-Opt$ moves but tries to find the best choice of k for each move.

To obtain a locally optimal solution, the search operation is confined among the neighbors of an initial solution, called neighborhood, where the neighbors are iteratively obtained through a systematic change of the nodes of the initial tour. Through this change, and by means of local search, an algorithm has been developed that leads to one kind of metaheuristic algorithm, known as variable neighborhood search (*VNS*) algorithm [27], [30]. The *VNS-1* and the *VNS-2* algorithms are recognized as such neighborhood search approaches, where the local searches (i.e., improvements over the present solution) are carried out through $2-Opt$ and quicker $2-Opt$ algorithms, respectively.

The objective of the present work is to suggest some modifications over the variable neighborhood search (*VNS*) algorithm, suggested by Hansen and Mladenović [27], [30] to solve *TSP*, with regard to the following three aspects:

- a reasonably good and computationally efficient choice of the initial tour,
- a clever choice of neighborhood construction,
- an incorporation of a stochastic approach, close to the more popular annealing schedule, to avoid ending at a local optimum and achieving the global optimal solution.

Through the above three aspects, the present hybrid *VNS* algorithm aims to deal with some imprecision and uncertainty in order to achieve near-optimal, robust and low-cost solution. These contributions have been attempted and highlighted by demonstrating the distributional properties of the final tour lengths obtained by the proposed algorithm and various other algorithms. The application aspect of the proposed algorithm is discussed through its ability to handle moderately large *TSP* with little computational complexity and at the same time its flexibility to take care of other areas of discrete optimization with equal competence. Such an algorithm has been suggested by Hore et al. [31, 32] to find a

near-optimal or optimal allocation design with different treatments to several experimental units with the known covariate(s). The proposed algorithm is similar to the algorithm suggested in the context of the design issue addressed there, with the necessary modifications of enlargement of the search space.

The paper has been structured as follows. In Section 2, a brief review of the *VNS* algorithm has been done. In order to obtain a near-optimal tour, a moderately acceptable initial solution is needed. A procedure to obtain such initial solution is discussed in Section 3. Section 4 describes the proposed algorithm, based on the initial tour. A detailed analysis of two *TSPs*, one each for the symmetric and asymmetric case, from the *TSPLIB* dataset of *R*, along with two more asymmetric *TSPs* generated through simulation experiments, are considered in Section 5. These examples establish the efficacy of the proposed algorithm in comparison to existing algorithms, as available and implemented in *R* and other *VNS* algorithms. The performance of the present algorithm is also judged in light of 60 benchmark symmetric *TSPs* from the *TSPLIB* dataset. The datasets and corresponding optimum solutions are available at [35]. To demonstrate the flexibility of the proposed hybrid *VNS* algorithm for solving combinatorial optimization problems, some applications other than the *TSP* are described in Section 6. The paper ends with some concluding remarks and scopes for further research.

2. Review of the *VNS* Algorithm

We start with a brief review of the variable neighborhood search (*VNS*) algorithm with special emphasis on *VNS* – 1 and *VNS* – 2, as proposed in [27], [30]. Consider the situation where a finite number k_{max} of pre-selected ordered neighborhood structures is available and let the set of neighbors in the k^{th} neighborhood of the tour x be denoted by $\mathcal{N}_k(x)$, for $k = 1, 2, \dots, k_{max}$. Note that the neighborhood search is permitted up to k_{max} number of times. The basic *VNS* may be summarized as follows and for details we refer to [27], [30].

Initialization: Find an initial solution x ; select the set of neighborhood structures $\mathcal{N}_k(x)$, $k = 1, 2, \dots, k_{max}$, that will be used in the search; choose a stopping condition;

Repeat the following until the stopping condition is met:

(1) *Set* $k \leftarrow 1$;

(2) *Until* $k = k_{max}$, *repeat the following steps* :

- (a) *Shaking*. Generate a point x' at random from the k^{th} neighborhood of x ($x' \in \mathcal{N}_k(x)$).
- (b) *Local Search*. Apply some local search method with x' as the initial solution; denote the so obtained local optimum by x'' .
- (c) *Move or Not*. If this local optimum is better than the incumbent, move there ($x \leftarrow x''$), and continue the search with $\mathcal{N}_1(x'')$ for $k \leftarrow 1$; otherwise set $k \leftarrow k + 1$.

Stopping Condition: Stop if maximum number of iterations is achieved.

The stopping condition may be based on the maximum amount of CPU time allowed, or the maximum number of iterations permitted, or the number of iterations between two significant improvements, and so on. In the simulation study and real life experiments, we have considered the maximum number of iterations as the stopping condition. The most popular version of local search algorithm for solving *TSP* is the *2-opt* algorithm. The basic *VNS* rules using *2-Opt* and *Quicker 2-Opt* as local search method are known as *VNS-1* and *VNS-2*, respectively. Thus, the only difference between *VNS-1* and *VNS-2* lies in the choice of the local search method, although the definition of neighborhood for both *VNS-1* and *VNS-2* are the same. Here $\mathcal{N}_k(x)$ is the k^{th} neighborhood of a tour x , for $k = 1, 2, \dots, n-1$, which is obtained by randomly exchanging $(k+1)$ elements of x , i.e., having $(k+1)$ edges different from x . In the simulation study and real life examples, we have observed that *VNS-2* consistently outperforms *VNS-1* with varying extent.

There is scope for improvement in both *VNS-1* and *VNS-2* with respect to three important aspects, namely, (1) determination of the initial tour, (2) construction of neighborhood structure including the method of local search and (3) developing an efficient stopping rule. In the next two sections, we suggest some improvements in these three aspects and, in Section 5, illustrate them to find optimal or near-optimal tour by means of some real-life examples and simulation studies. Apart from solving the *TSP*, the hybrid *VNS* algorithm has also been applied in the fields of optimal design of experiments [31, 32], wireless sensor networks [33] and reliability theory and survival analysis [34], and such applications are briefly mentioned in Section 6.

3. The Initial Tour

We propose a greedy algorithm to obtain an initial solution, which is intuitively very simple and quite similar to *NNA*. In *NNA*, a tour on the basis of a subset of randomly chosen cities adds one city which is not included in the tour and is closest to the last city of the existing tour. Here we suggest that the tour should start from that particular sub-tour with two cities which has the least distance among all such possible sub-tours. Although such algorithm usually does not give the global optimum solution, we may consider it as the initial solution [36].

Let $I = \{1, 2, \dots, n\}$ be the set of n distinct cities. The cost function between any two cities r and s is given by $d(r, s)$ with $d(r, r) = 0$, for $r, s \in I$. For symmetric *TSP*, $d(r, s) = d(s, r)$, for all $r, s \in I$. Suppose $d(i, j) = \min_{r,s} \{d(r, s); r, s \in I\}$ and assume this to be unique; if not, one can choose the minimum randomly. The tour should start from i to j , denoted by $\{i, j\}$. Let k be the nearest city with the tour $\{i, j\}$, in terms of the minimum cost $d(i, j, k)$ or $d(k, i, j)$ depending upon whichever is minimum, where $d(i, j, k) = \min_l \{d(i, j) + d(j, l), l \in I, l \neq i, j\}$ or $d(k, i, j) = \min_l \{d(l, i) + d(i, j), l \in I, l \neq i, j\}$, respectively. Then the city k is augmented with the already existing tour $\{i, j\}$ either at the beginning or at the end, and the augmented tour is denoted by either of $\{k, i, j\}$ or $\{i, j, k\}$, respectively. The procedure is continued till all the cities are included in the tour.

A good initial solution with less computational effort is the key to arrive at an optimal or near-optimal solution through any kind of metaheuristic algorithm. Otherwise, an inferior choice of the initial solution may mislead the entire optimization approach to a local optimum and hence an inferior final solution. We have proposed a simple method to obtain an initial solution with the minimum computational burden. According to Hansen and Mladenović ([27], [30]), the most popular heuristic to solve *TSP* is *2-opt* algorithm. Thus, it might be a good choice of initial solution, but needs extensive computation, whereas our suggested initial method requires lesser computational burden with no uncertainty. A comparative study between the suggested initial and the initial obtained from the *2-opt* algorithm for moderately large number of cities with regard to their abilities to converge to the optimal or near-optimal solution considering various examples from *TSPLIB* dataset is reported in Section 5. Some graphical comparisons are also reported through Figures 1-3. For obtaining the optimal or near-optimal solution with the suggested initial solution, an extension of the

Once the initial tour is judiciously obtained, an iterative algorithm is proposed to arrive at the optimal or near-optimal tour. Consider a tour given by a permutation $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n)$ of $\{1, 2, \dots, n\}$, the labels of n cities. Let us define the first order neighborhood of a tour α , denoted by $\mathcal{N}_1(\alpha)$, as $\{\alpha_{(11)}, \alpha_{(12)}, \dots, \alpha_{(1n)}\}$, where $\alpha_{(1i)}$ is the permutation obtained from α by exchanging the elements at the i^{th} and $(i+1)^{st}$ locations, i.e. α_i and α_{i+1} , respectively, for $i = 1, 2, \dots, n$, with $\alpha_{n+1} = \alpha_1$, keeping other elements unchanged. Thus,

$$\begin{aligned}\alpha_{(11)} &= (\alpha_2, \alpha_1, \alpha_3, \dots, \alpha_{n-1}, \alpha_n), \\ \alpha_{(12)} &= (\alpha_1, \alpha_3, \alpha_2, \dots, \alpha_{n-1}, \alpha_n), \\ &\dots\dots\dots \\ \alpha_{(1 \overline{n-1})} &= (\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_n, \alpha_{n-1}), \\ \alpha_{(1 \overline{n})} &= (\alpha_n, \alpha_2, \alpha_3, \dots, \alpha_{n-1}, \alpha_1).\end{aligned}$$

The neighborhood construction for obtaining the neighbors of a selected initial tour may be generalized on the basis of this location exchanging concepts in subsequent orders of neighborhood. The k th order neighborhood of a tour α , denoted by $\mathcal{N}_k(\alpha)$, is constructed by exchanging α_i and $\alpha_{i+k \pmod n}$, for $i = 1, 2, \dots, n$, while keeping other elements unchanged. Note that $\mathcal{N}_i(\alpha) = \mathcal{N}_{n-i}(\alpha)$, for $i = 1, 2, \dots, n$, so that maximum value of k is $k_{max} = \lfloor n/2 \rfloor$. The cost of the tour corresponding to the permutation α is denoted by $V(\alpha)$, where $V(\alpha) = d(\alpha_1, \alpha_2) + d(\alpha_2, \alpha_3) + \dots + d(\alpha_{n-1}, \alpha_n) + d(\alpha_n, \alpha_1)$. All the $n!$ possible tours are connected through this concept of neighborhood in the sense that any tour may be obtained from any another tour through a finite number of such exchange operations. The proposed algorithm for obtaining the optimum tour, starting from an initial tour, is described in the following steps along with explanation after each step.

Step 1: Start with the initial tour $\alpha^{(0)}$ with corresponding cost $V(\alpha^{(0)})$, say.

The initial tour is selected by the algorithm suggested in Section 3. The performance of the proposed algorithm depends upon a reasonable choice of the initial tour, failing which

one may end up with one that is far from the optimal tour.

Step 2: Consider $k = 1$.

The exploration of neighborhoods starts with the first order neighbors of the initial tour.

Step 3 : Consider all the k th order neighbors in $\mathcal{N}_k(\alpha^{(0)})$, and compute $V(\alpha')$, for all $\alpha' \in \mathcal{N}_k(\alpha^{(0)})$.

Note that, for each k , a n -dimensional vector represents a neighbor and there are n such different neighbors. The cost for each of them is computed.

Step 4: If $\min \{V(\alpha'), \alpha' \in \mathcal{N}_k(\alpha^{(0)})\} < V(\alpha^{(0)})$, choose the next improved tour to be $\alpha^{(1)} = \arg \min \{V(\alpha'), \alpha' \in \mathcal{N}_k(\alpha^{(0)})\}$, and replace $\alpha^{(0)}$ by $\alpha^{(1)}$, to start search again from Step 2.

A better tour than the initial one is found from among the k th order neighbors and the algorithm starts afresh from Step 2, with the better tour replacing the initial one, to find a still better tour than the present one.

Step 5: Otherwise, set $k = k + 1$ and go to Step 3, till $k = \lfloor n/2 \rfloor$.

If no better tour is found until the k th order, then extend the search domain to include the $(k + 1)$ st order neighbors of $\alpha^{(0)}$ and the search is continued till a better tour is obtained from among all the neighbors of the previous neighborhoods and including the present one.

Step 6: When $k > \lfloor n/2 \rfloor$, check the stopping condition described in Step 7. If it is not satisfied, choose $\alpha^{(1)}$ randomly from $\alpha^{(0)} \cup \mathcal{N}_1(\alpha^{(0)}) \cup \dots \cup \mathcal{N}_{\lfloor n/2 \rfloor}(\alpha^{(0)})$ with probabilities given by $p_0 = \frac{V^{-1}(\alpha^{(0)})}{D(\alpha^{(0)})}$ at $\alpha^{(0)}$, $p_{\alpha'} = \frac{V^{-1}(\alpha')}{D(\alpha^{(0)})}$, for $\alpha' \in \mathcal{N}_1(\alpha^{(0)}) \cup \dots \cup \mathcal{N}_{\lfloor n/2 \rfloor}(\alpha^{(0)})$, where $D(\alpha^{(0)}) = V^{-1}(\alpha^{(0)}) + \sum_{\alpha' \in \mathcal{N}_1(\alpha^{(0)}) \cup \dots \cup \mathcal{N}_{\lfloor n/2 \rfloor}(\alpha^{(0)})} V^{-1}(\alpha')$. Set $\alpha^{(0)} = \alpha^{(1)}$ and go to Step 2.

If no better tour is found till Step 5 with $k = \lfloor n/2 \rfloor$, a tour is selected randomly with preassigned probabilities from among all the tours already searched in all the $\lfloor n/2 \rfloor$ neighborhoods, including the initial tour, and the algorithm is started afresh from Step 2. The preassigned probabilities are computed so as to put smaller chances of being selected as the next tour for those having larger cost. From the above-described method, it is clear that the algorithm may get stuck at a local minimum if there is no better alternative among the searched neighborhoods. A remedy in this regard may be suggested as to restart the algorithm with another initial tour chosen randomly from among all the searched neighborhoods. This may not entirely solve the problem of local optimum, but has been found to be effective

for the search of the global optimum.

Step 7: The algorithm continues till a solution reached with some desired accuracy by satisfying some convergence criterion.

This accuracy may be achieved in several ways, for example, by limiting the number of returns, say 10, to a certain tour. In the present paper, we have generalized the approach mentioned in [31], by suggesting a probabilistic method, as follows. At the i^{th} return ($i \geq 1$) to a tour $\alpha^{(0)}$, the preassigned probabilities p_0 and $p_{\alpha'}$, respectively, for choosing $\alpha^{(0)}$ and α' in $\mathcal{N}_1(\alpha^{(0)}) \cup \dots \cup \mathcal{N}_{[n/2]}(\alpha^{(0)})$, as described in Step 6 above, are modified as $p_0^{(i)} = \frac{V^{(i)}(\alpha^{(0)})}{D^{(i)}(\alpha^{(0)})}$, $p_{\alpha'}^{(i)} = \frac{V^{(i)}(\alpha')}{D^{(i)}(\alpha^{(0)})}$ with $D^{(i)}(\alpha^{(0)}) = V^{(i)}(\alpha^{(0)}) + \sum_{\alpha' \in \mathcal{N}_1(\alpha^{(0)}) \cup \dots \cup \mathcal{N}_{[n/2]}(\alpha^{(0)})} V^{(i)}(\alpha')$, $V^{(i)}(\alpha^{(0)}) = V^{-1}(\alpha^{(0)}) + (D(\alpha^{(0)}) - V^{-1}(\alpha^{(0)})) \times \frac{i}{n^{[n/2]}}$, $V^{(i)}(\alpha') = \max \{V^{-1}(\alpha') - (D(\alpha^{(0)}) - V^{-1}(\alpha^{(0)})) \times \frac{i}{n^{[n/2]}}, 0\}$, for $\alpha' \in \mathcal{N}_1(\alpha^{(0)}) \cup \dots \cup \mathcal{N}_{[n/2]}(\alpha^{(0)})$. This increases the probability of choosing the same tour $\alpha^{(0)}$ at successive returns. We suggest stopping the algorithm when this probability exceeds a preassigned value close to unity, usually taken as 0.95 or 0.99.

The VNS algorithm for a minimization problem, as introduced in [27], is a descent method, while the proposed method is a descent-ascent method with the inclusion of Step 6. This is because, even if a new solution is worse than the present one, a better solution might be obtained from among all the neighbors of the worse tour in any subsequent step. The concept of such descent-ascent method is motivated from the well-known simulated annealing (SA) method [21], where the similarity lies in the selection of a new solution from among all its neighbors through some stochastic mechanism. The proposed algorithm, including the stochastic approach, may be summarized as follows.

Initialization: Obtain the initial tour $\alpha^{(0)}$ using the method of Section 3.

Repeat the following procedure until the stopping condition is achieved:

- (1) Set $k \leftarrow 1$;
- (2) Until $k = [n/2]$, repeat the following steps :
 - (a) *Construction of Neighborhood:*

Construct the k th order neighborhood $\mathcal{N}_k(\alpha^{(0)})$ of $\alpha^{(0)}$, and compute the cost $V(\alpha')$ corresponding to the tour α' , for $\alpha' \in \mathcal{N}_k(\alpha^{(0)})$

- (b) *Exploration of Neighborhood:*

Find the best neighbor $\alpha^{(1)}$ of $\alpha^{(0)}$ by satisfying $V(\alpha^{(1)}) = \arg \min \{V(\alpha'), \alpha' \in \mathcal{N}_k(\alpha^{(0)})\}$

(c) *Move or Not* :

If the solution $\alpha^{(1)}$ is better than $\alpha^{(0)}$, move there by setting $\alpha^{(1)} \leftarrow \alpha^{(0)}$ and go to (1) to continue the search in $\mathcal{N}_1(\alpha^{(0)})$; otherwise set $k \leftarrow k + 1$.

(3) When $k > \lceil n/2 \rceil$, check the stopping condition. If the condition is not satisfied choose a tour $\alpha^{(1)}$ randomly with preassigned probabilities among all the neighbors in $\alpha^{(0)} \cup \mathcal{N}_1(\alpha^{(0)}) \cup \dots \cup \mathcal{N}_{\lceil n/2 \rceil}(\alpha^{(0)})$; then set $\alpha^{(1)} \leftarrow \alpha^{(0)}$ and go to (1).

Stopping Condition:

Until the probability of returning to the same tour is above 0.95 or 0.99.

The performance of the hybrid *VNS* algorithm in comparison to the existing algorithms are assessed through various simulation studies and real-life examples and are reported in the next section.

5. Illustrations

In this section, we investigate the performance of the proposed algorithm in comparison with the various other algorithms available in R. We consider two examples, namely USCA50 and ft53, for symmetric and asymmetric *TSPs*, respectively, from the *TSPLIB* dataset of R. We also consider two asymmetric *TSPs* with 10 and 50 cities, respectively, where the distances between the cities are simulated from an *exponential* distribution with mean = 25. The performance of the proposed algorithm is compared with *NNA*, *RNNA*, *2-Opt*, *FI*, *NI*, along with the *VNS-1* and the *VNS-2* algorithms by considering the two real life as well as the two simulated datasets, as described before. The efficiency of the proposed algorithm over any of the above seven algorithms is defined by the ratio of the values of the cost for the tour obtained by the particular competing algorithm and the same for the tour obtained by the proposed algorithm. Formally, the efficiency is defined as $\frac{V(\alpha^*)}{V(\alpha^P)}$, with α^* and α^P denoting the tours obtained by the competing and the proposed algorithm, respectively. The efficiency values may be multiplied by 100 to express them in percentage. As the proposed algorithm has an in-built stochastic component, the computation of the efficiency value is repeated 1000 times and the average, along with the minimum and maximum efficiency values, are reported, for each of the seven cases. Tables 1 and 2, respectively, present the findings of USCA50 and ft53 datasets, while Tables 3 and 4 report the findings from the

simulation experiments on the basis of 10 and 50 cities, respectively.

For the USCA50 dataset, which is an example of symmetric TSP , the mean efficiencies of the proposed algorithm with respect to the seven competing algorithms are greater than 100% indicating the superiority of the proposed algorithm over those. However, the gain in average efficiency of the proposed algorithm over the $2 - Opt$, FI , $VNS - 1$, $VNS - 2$ algorithms are marginal compared to those over the rest. While comparing with these four algorithms, the minimum efficiency gain of the proposed algorithm over 1000 such repetitions falls marginally below 100% indicating that, in some situations, these algorithms may outperform the proposed one, albeit to a small extent. For the ft53 dataset, which is an asymmetric TSP , the gain in average efficiency of the proposed algorithm over all the seven competing algorithms is found to be moderate to very high, with an increase by 9 to 58%. Moreover, the minimum efficiency values for all other competing algorithms, except the $VNS - 2$ algorithm, are found to be larger than 100%. This indicates that in the worst case also, the proposed algorithm possibly outperforms all except the $VNS - 2$ algorithm, in which case also the efficiency loss is marginal. Thus the performance of the proposed algorithm over the seven competing algorithms may be claimed to be marginal to moderately superior for an asymmetric TSP .

For the two simulated examples of asymmetric TSP , the mean efficiency is never less than 100% and sometimes may be as high as 200%. Although in some cases, the minimum efficiency in comparison to $2 - opt$, $VNS - 1$ and $VNS - 2$ are less than 100%, the loss in minimum efficiency is always marginal.

< Approximately Table 1 is here >

< Approximately Table 2 is here >

< Approximately Table 3 is here >

< Approximately Table 4 is here >

The distributional property resulting from the choice of the initial tour in comparison to the more popular $2 - opt$ approach is studied by constructing histogram of the final tour length as obtained over 1000 different iterations. Such histograms of the final solutions, for obtaining the optimal or near-optimal tour by the $2 - opt$ algorithm, the $VNS - 2$ algorithm with the final solution of the $2 - opt$ algorithm as the initial tour and the proposed hybrid VNS with the suggested initial tour have been considered. The objective of drawing such

histogram is to show the consistency and accuracy of an algorithm, for finding the optimal or near-optimal tour starting from the respective initial tour. Three instances (viz. *krob100*, *kroa200* and *rd400*) from *TSPLIB* dataset are considered here to show the convergence from initial to final optimal or near-optimal tour. The *VNS-1* is not considered owing to its inferiority to the *VNS-2* algorithm as reported in [27], [30]. In Figure 1, for *krob100* data, the histograms of the distribution of the final tour lengths as obtained by the *2-opt* algorithm, *VNS-2* algorithm and the proposed hybrid *VNS* algorithm, are given. The tour length corresponding to the suggested initial tour is also marked. In each case, the respective algorithm has been repeated 1000 times and the histogram of the corresponding final tour lengths are drawn. It may be noted that our initial algorithm is non-random, yielding the same tour length for a given problem, while our proposed algorithm predominantly converges to an optimal tour with same tour length, which is found to be better than the final solutions of the other algorithms. Note that such final solutions obtained by the *2-opt* and *VNS-2* algorithms are not that consistent, yielding various optimum tour lengths over several iterations. The performance of the *VNS-2* algorithm has been found comparatively better than the *2-opt* in the sense of lower tour length and narrower range of the optimal solutions. It has also been noticed that the *2-opt* algorithm performs uniformly better than our suggested initial. The main advantage of the proposed hybrid *VNS* lies in its ability to yield better solution than the *2-opt* and the *VNS-2* algorithms even when the starting point is relatively inferior. The same exercise has been undertaken for two other datasets, viz. *kroa200* and *rd400*, and with comparison through the histograms, as shown in Figures 2 and 3, the efficacy and the relative performances of the algorithms are assessed. The nature of findings has been found to be almost similar. Among 1000 iterations, the proposed hybrid *VNS* algorithm performs uniformly better than the *VNS-2* and the *2-opt* algorithms.

< Approximately Figure 1 is here >

< Approximately Figure 2 is here >

< Approximately Figure 3 is here >

Following Geng et al. [37], we also obtain the best, the average and the worst cost values out of 1000 replications of the proposed algorithm for some of the benchmark problems of the *TSPLIB* datasets. An error in percentage for each instance, computed on the basis of the percentage of the deviation of the best value as obtained through the proposed al-

gorithm from the optimum value as reported for the said dataset, has also been computed. The corresponding CPU times (in second) have also been noted. The results are given in Table 5. We find that the loss of efficiency in the best solution, in comparison with the optimum one, as reported in the *TSPLIB* dataset, is mostly within 10% and seldom exceeds that, specifically for problems with a larger number of cities. Moreover, in some instances, the proposed algorithm gives better solutions (for example, for *krob100*, *pr107* and *pr144* datasets) than those reported in the corresponding *TSPLIB* dataset, with negative error percentage entries in the table. The entire computation for the proposed algorithm is carried out in R software version 3.3.1 with a modest supporting system **AMD Phenom(tm) II N930 Quad-Core Processor 2.00 GHz** with 4.00 GB RAM in a standard laptop.

< Approximately Table 5 is here >

The hybrid *VNS* algorithm for solving the *TSP*, has been successfully implemented in several other application areas and these applications are briefly described in Section 6.

6. Some Other Applications

The hybrid *VNS* algorithm, as introduced in the present paper, is an extension of the usual *VNS* algorithm, where the improvement from one solution to its neighbor is done through a stochastic step, similar to the annealing schedule. Such stochastic step enables the algorithm to escape a local optimality and attain the actual or approximate global optimal solutions. We have discussed the applicability of this hybrid discrete optimization procedure in three other problems, where the issue of discrete optimization is to be addressed. This ensures the versatility and applicability of the proposed procedure in various other complex problems, apart from its ability to solve the conventional *TSP* with moderate to high computational complexity.

6.1 Optimal Design of Experiments

Finding the optimal design for allocating two or more treatments to a fixed group of experimental units with several known covariates is an important problem in many scientific research and has been extensively studied by Hore et al. [31,32]. The design problem here is to determine an optimal allocation rule for the experimental units with known covariate(s)

into multiple treatments such that both treatment and covariate effects, or only treatment effect, are efficiently estimated in terms of the commonly used D - or A -optimality. The actual search may be judiciously confined to a smaller number of schemes leading to an optimal solution, while finding the exact optimal allocation design is generally a computationally challenging issue, especially when the number of experimental units and the number of covariates are large. However, a computationally tractable algorithm with lesser amount of computer time is always preferred to achieve an optimal or near-optimal solutions for such problems. In this context, this improved hybrid *VNS* algorithm, with hybridization of the stochastic approach, has been effectively developed and implemented in [31, 32]. The algorithm generates efficient allocation in comparison to random allocation and other heuristics allocation rules suggested by Harville [38], which have been demonstrated through simulation experiments and real-life data analysis. In the process, the computational complexity could be reduced to a great extent.

6.2 Wireless Sensor Network

Sensor networks aim at monitoring their surroundings for event detection and object tracking. Owing to failure or death of sensors, false signal may be transmitted. The distributed fault detection in wireless sensor network (*WSN*) has been considered as problem of interest in Nandi et al. [33]. In particular, they have considered how to take decision regarding fault detection in a noisy environment as a result of false detection or false response of event by some sensors, where the sensors are placed at the center of regular hexagons and the event can occur at only one hexagon. The fault detection schemes explicitly introduce the error probabilities into the optimal event detection process. Two types of detection probabilities, one for the center node, where the event occurs, and the other one for the adjacent nodes has been introduced. This second type of detection probability is new in sensor network literature. The construction of the adjacent nodes corresponding to center node may be framed by the suggested neighborhood structure and the selection of any of the adjacent node may be designed through the hybrid *VNS* procedure. It may be noted that the total number of such nodes in the *WSN* is extremely large, thus requiring some systematic optimization procedure to arrive at the best solution. The researchers have developed schemes under the model selection procedure and multiple model selection procedure

and use the concept of Bayesian model averaging to identify a set of likely fault sensors and obtain an average predictive error.

6.3 Reliability Theory and Survival Analysis

In the determination of optimum Type-II progressive censoring scheme, the experimenter needs to carry out an exhaustive search within the set of all admissible censoring schemes. The existing recommendations are only applicable for small sample sizes. The implementation of exhaustive search techniques for large sample sizes is not feasible in practice. The hybrid *VNS* algorithm may be invoked to overcome this computationally challenging issue, especially for large sample sizes. It is also noted, that the algorithm gives exactly the same solution as obtained through the exhaustive search, where it is possible to implement, i.e. for small sample set up. A cost function-based optimality criterion has been proposed in [34], which is scale invariant for location-scale and log-location-scale families of distributions. This approach accommodates the constrained optimization set up in the hybrid *VNS* procedure. A sensitivity analysis is also carried out to study the effect of misspecification of parameter values or cost coefficients on the optimum solution.

7. Concluding Remarks

The present paper proposes a *VNS* algorithm with an inbuilt stochastic search approach for solving both symmetric as well as asymmetric *TSPs*, with equal ease of computation. While the performance of the algorithm seems to be marginally better than the presently available methods for symmetric problems, a significant improvement is observed for asymmetric cases. The average computation times for the proposed algorithm with the already mentioned computational strength for the 60 symmetric *TSP* benchmark problems are noted. All the standard algorithms are primarily developed to deal with symmetric *TSPs* and the asymmetric *TSPs* are reformulated as symmetric *TSPs* doubling the number of cities by introducing a dummy city corresponding to each city [39]. The main advantage of the proposed algorithm is its applicability to the asymmetric problems with equal flexibility, without converting the asymmetric problem into a symmetric one. The adaptive simulated annealing algorithm with greedy search (*ASA – GS*) for the *TSP* has been developed in [37] that

has achieved better results with faster convergence for large-scale problems of the *TSPLIB* dataset. But, the method has been framed for only symmetric *TSPs*, while the proposed algorithm has been framed for a general *TSP* format. It is important to note that the present algorithm yields a better solution for a few symmetric problems given in *krob100*, *pr107*, and *pr144*, in comparison to those reported in the *TSPLIB* dataset. A modified simulated annealing (*SA*) algorithm, named as list-based simulated annealing (*LBSA*) algorithm has been developed for solving symmetric *TSPs* in [40] by incorporating a selected list-based cooling schedule to obtain an optimal or near-optimal tour, with faster convergence rate. However, the *LBSA* algorithm also fails to achieve better optimum values for the above three datasets [40]. The flexibility of the proposed algorithm to solve some other optimization problems, apart from the *TSP*, arising in several other disciplines has been mentioned. This indicates that the hybrid *VNS* is an interesting extension of the usual *VNS* and has the capability to be implemented in various other fields.

Till date, it is generally believed that Concorde [41, 42] is possibly the best available algorithm for solving symmetric *TSP*'s based on the Branch-and-Cut [43] and Branch-and-Bound [44] method. The optimal tour value of USCA50 and ft53 data from the *TSPLIB* dataset by using Concorde are reported in [24, 45], respectively. Although the average performance of the proposed algorithm has been found to be marginally inferior to Concorde, its performance has been found to be equally effective as Concorde in some replications. The detailed description and steps of the Concorde algorithm are not available in public domain and the software has to be installed separately, governed by a different license [42] and may not be compatible with all systems.

Applegate et al. [41] established new breakthrough results for the *TSP*. For instance, the research team solved an instance of the *TSP* of 13,509 cities corresponding to all US cities with populations of more than 500 people. The approach involved ideas from polyhedral combinatorics and combinatorial optimization, integer and linear programming, computer science data structures and algorithms, parallel computing, software engineering, numerical analysis, graph theory, and more. The solution of this instance demanded the use of three Digital Alpha Server 4100s (with a total of 12 processors) and a cluster of 32 Pentium-II PCs. The complete calculation took approximately three months of computer time. The code had more than 1,000 pages and is based on state-of-the-art techniques from a wide variety of scientific fields [46]. This speaks of the volume of labor and effort needed to solve a large

TSP. However, our method has the ability to tackle moderately large *TSPs* and to give relatively efficient solution by consuming very limited computer time in a standard laptop, without concerning about the problem being symmetric or asymmetric. At the same time, the present algorithm is intuitively very simple and may be judiciously implemented to solve various other problems of discrete optimization coming from different fields, as have been discussed [31–34]. Some interesting extensions of the existing *TSPs* might be formulated and may be solved by suitably modifying the proposed algorithm in a constrained setup. One such extension might be assigning some infinitely large values in the cost matrix, thus blocking the corresponding paths in the tour. Another possible extension is to allow multiple visits to a selected group of cities, either as an option or as a mandate. The *TSP* with profits [47] may be an interesting extension, where it is not necessary to visit all the vertices and a profit is associated with each vertex. Extension of the algorithm to a multi-dimensional *TSP*, e.g. the spherical *TSP*, in which all the nodes (cities) and paths (solutions) are located on the surface of a sphere [48], might be an interesting problem. Works in these directions are already in progress and would be reported elsewhere.

Acknowledgment

The authors like to thank Michael Hahsler, Southern Methodist University, USA and Kurt Hornik, Wirtschaftsuniversität Wien, Austria for some helpful discussions and their R - guide notes ‘*TSP*-Infrastructure for the Traveling Salesperson Problem and Package *TSP*’. The authors are also grateful to William Cook, Professor in Combinatorics and Optimization, University of Waterloo for his valuable suggestions and his reference at www.iwr.uni-heidelberg.de/groups/comopt/index.html. The authors are also thankful to the Associate Editor and two anonymous reviewers for their valuable comments, which improved the presentation of the paper to a considerable extent.

References

- [1] Z. C. S. S. Hlaing, M. A. Khine, Solving traveling salesman problem by using improved ant colony optimization algorithm, *International Journal of Information and Education Technology* 1 (5) (2011), 404-409.

- [2] Y. Marinakis, M. Marinaki, G. Dounias, A hybrid particle swarm optimization algorithm for the vehicle routing problem, *Engineering Applications of Artificial Intelligence* 23 (2010), 463-472.
- [3] K. Savla, E. Frazzoli, F. Bullo, Traveling salesperson problems for the Dubins vehicle, *IEEE Transactions on Automatic Control* 53 (6) (2008), 1378-1391.
- [4] M.K. Mehmet Ali, F. Kamoun, Neural networks for shortest tour computation and routing in computer networks, *IEEE Transactions on Neural Network* 4 (5) (1993), 941-953.
- [5] D. Banaszak, G.A. Dale, A.N. Watkins, J.D. Jordan, An optical technique for detecting fatigue cracks in aerospace structures, *Proceeding of 18th ICIASF*, (1999) pp. 1-7.
- [6] C.H. Papadimitriou, Euclidean traveling salesman problem is NP-complete, *Theoretical Computer Science* 4 (1978), 237-244.
- [7] M. R. Garey, D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W.H. Freeman : New York, 1979.
- [8] R. L. Graham, *Handbook of Combinatorics*, MIT Press, 1995.
- [9] G. Finke, A. Claus, E. Gunn, A two-commodity network flow approach to the traveling salesman problem, *Congressus Numerantium* 41 (1984), 167-178.
- [10] R. Bellman, Dynamic programming treatment of the travelling salesman problem, *Journal of ACM*, 9 (1) (1962), 61-63.
- [11] M. Held and R. M. Karp, A dynamic programming approach to sequencing problems, *Journal of Society for Industrial and Applied Mathematics* 10 (1) (1962), 196-210.
- [12] J. Kennedy, R. C. Eberhart, Particle swarm optimization, *Proceedings of the IEEE International Conference on Neural Networks IV* (1995), 1942-1948, <http://dx.doi.org.10.1109/ICNN.1995.488968>.
- [13] Y. Shi, R. C. Eberhart, A modified particle swarm optimizer, *Proceedings of IEEE International Conference on Evolutionary Computation* (1998), 69-73.

- [14] Y. Marinakis, M. Marinaki, A hybrid multi-swarm particle swarm optimization algorithm for the probabilistic traveling salesman problem, *Computers and Operations Research* 37 (2010), 432-442.
- [15] T. A. S. Masutti, L. N. de Castro, A self-organizing neural network using ideas from the immune system to solve the traveling salesman problem, *Information Sciences* 179 (10) (2009), 1454-1468.
- [16] M. Dorigo, V. Maniezzo, A. Coloni, Ant system: optimization by a colony of cooperating agents, *IEEE Transactions on Systematic, Man and Cybernetics, Part B : Cybernetics* 26 (1) (1996), 29-41.
- [17] S. Ghafurian, N. Javadian, An ant colony algorithm for solving fixed destination multi-depot multiple traveling salesman problems, *Applied Soft Computing* 11 (1) (2011), 1256-1262.
- [18] D. Whitley, A genetic algorithm tutorial, *Statistics and Computing* 4 (1994), 65-85.
- [19] R. Baraglia, J.I. Hidalgo, R. Perego, A hybrid heuristics for the traveling salesman problem, *IEEE Transactions on Evolutionary Computation* 5 (6) (2001), 613-622.
- [20] L. Jiao, L. Wang, A novel genetic algorithm based on immunity, *IEEE Transactions on Systems, Man and Cybernetics Part A: Systems and Humans* 30 (5) (2000), 826-830.
- [21] S. Kirkpatrick, C. D. Gelatt, M. Vecchi, Optimization by simulated annealing, *Science* 220 (4598) (1983), 671-680.
- [22] V. Cerny, Thermodynamical approach to the traveling salesman problem: an efficient simulation algorithm. *Journal of Optimization Theory and Applications* 45 (1985), 41-51.
- [23] D. J. Rosenkrantz, R. E. Stearns, M. L. I. Philip, An analysis of several heuristics for the traveling salesman problem, *SIAM Journal on Computing* 6(3) (1977), 563-581.
- [24] M. Hahsler, K. Hornik, *TSP* - Infrastructure for the traveling salesperson problem, *Journal of Statistical Software* 23 (2) (2007), 1-21.

- [25] G. A. Croes, Method for solving traveling-salesman problems, *Operations Research* 6 (6) (1958), 791-812.
- [26] M. M. Flood, The traveling salesman problem, *Operations Research* 4 (1956), 61-75.
- [27] P. Hansen, N. Mladenović, An introduction to variable neighborhood search. in : S. Voss, S. Martello, I. Osman, and C. Roucairol, C. (Eds.), *Meta-heuristics: Advances and Trends in Local Search Paradigms for Optimization*, Kluwer Academic Publishers, MA, USA, 1999, 433-458.
- [28] S. Lin, Computer solutions of the traveling-salesman problem, *Bell System Technology Journal* 44 (1965) 2245-2269.
- [29] S. Lin, B. Kernighan, An effective heuristic algorithm for the traveling salesman problem, *Operations Research* 21(2) (1973), 498-516.
- [30] P. Hansen, N. Mladenović, Variable neighborhood Search : Principles and Applications, *European Journal of Operational Research* 130 (2001), 449-467.
- [31] S. Hore, A. Dewanji, A. Chatterjee, Design issues related to allocation of experimental units with known covariates into two treatment groups, *Journal of Statistical Planning and Inference* 155 (2014), 117-126.
- [32] S. Hore, A. Dewanji, A. Chatterjee, On optimal allocation of experimental units with known covariates into multiple treatment groups, *Calcutta Statistical Association Bulletin* 68 (1-2) (2016), 69-81.
- [33] M. Nandi, A. Dewanji, B. Roy, S. Sarkar, Model Selection Approach for distributed fault detection in wireless sensor networks, *International Journal of Distributed Sensor Networks* (2014), [http://dx.doi.org. 10.1155/2014/148234](http://dx.doi.org.10.1155/2014/148234).
- [34] R. Bhattacharya, B. Pradhan, A. Dewanji, On optimum life testing plans under Type-II progressive censoring scheme using variable neighborhood search algorithm, *TEST* 25(2) (2014) 309-330, <http://dx.doi.org.10.1007/s11749-015-0449-z>.
- [35] *TSPLIB* : <https://www.iwr.uni-heidelberg.de/groups/comopt/software/TSPLIB95/> (accessed 15 September 2017)

- [36] J. Bang-Jensen, G. Gutin, A. Yeo, When the Greedy Algorithm fails, *Discrete Optimization* 1(2) (2004), 121-127.
- [37] X. Geng, Z. Chen, W. Yang, D. Shi, K. Zhao, Solving the traveling salesman problem based on an adaptive simulated annealing algorithm with greedy search, *Applied Soft Computing* 11 (4) (2011), 3680-3689.
- [38] D. A. Harville, Nearly Optimal allocation of experimental units using observed covariate values. *Technometrics* 16 (4) (1974), 589-599.
- [39] R. Jonker, T. Volgenant, Transforming asymmetric into symmetric traveling salesman problems, *Operations Research Letters* 2 (1983), 161-163.
- [40] S. Zhan, J. Lin, Z. Zhang, Y. Zhong, List-based simulated annealing algorithm for traveling salesman problem, *Computational Intelligence and Neuroscience* <http://dx.doi.org/10.1155/2016/1712630>, (2016).
- [41] D. Applegate, R.E. Bixby, V. Chvatal, W. Cook, *TSP* Cuts which do not conform to the template paradigm. In M Junger, D Naddef (Eds.), *Computational Combinatorial Optimization, Optimal or Provably Near-Optimal Solutions*, 2241, *Lecture Notes in Computer Science*, Springer-Verlag, London, 2000, 261-304.
- [42] D. Applegate, R.E. Bixby, V. Chvatal, W. Cook, Concorde *TSP* Solver. <http://www.tsp.gatech.edu/Concorde/>, (2006).
- [43] M. Padberg, G. Rinaldi, Facet identification for the symmetric traveling salesman polytope, *Mathematical Programming* 47(2) (1990), 219 -257.
- [44] A. Land, A. Doig, An automatic method for solving discrete programming problems, *Econometrica* 28 (1960), 497-520.
- [45] Best known solutions for asymmetric *TSPs* : [http://comopt.ifi.uni-heidelberg.de /software/TSPLIB95/ATSP.html](http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/ATSP.html) (accessed 15 September, 2017)
- [46] P. Moscato, C. Cotta, *A Gentle Introduction to Memetic Algorithms Handbook of Metaheuristics*, Kluwer Academic Publishers, NY, 2003.

- [47] D. Feillet, P. Dejax, M. Gendreau, Traveling salesman problems with profits, *Transportation Science* 39 (2) (2005), 188-205.
- [48] X. Chen, Y. Zhoua, T. Zhonghua, L. Qifang, A hybrid algorithm combining glowworm swarm optimization and complete 2-opt algorithm for spherical travelling salesman problems, *Applied Soft Computing* 58 (September) (2017), 104-114.

Table Caption

Table 1. Efficiency of the tours obtained by proposed algorithm with respect to the different algorithms over 1000 repetitions for USCA50 data.

Table 2. Efficiency of the tours obtained by proposed algorithm with respect to the different algorithms over 1000 repetitions for ft53 data.

Table 3. Efficiency of the tours obtained by proposed algorithm with respect to the different algorithms over 1000 repetitions for $n = 10$ randomly selected cities.

Table 4. Efficiency of the tours obtained by proposed algorithm with respect to the different algorithms over 1000 repetitions for $n = 50$ randomly selected cities.

Table 5. Efficiency of the proposed algorithm for *TSP* benchmark instances from the *TSPLIB* data set.

Figure Caption

Figure 1. Histogram of final tour lengths for *krob100* data with regard to different algorithms.

Figure 2. Histogram of final tour lengths for *kroa200* data with regard to different algorithms.

Figure 3. Histogram of final tour lengths for *rd400* data with regard to different algorithms.

Highlights:-

- The ability to solve the Traveling Salesman Problem (*TSP*), in a computationally efficient way, is often considered to be the benchmark of any optimization algorithm.
- An improvement in the Variable Neighborhood Search (*VNS*) algorithm with a stochastic approach has been suggested. This algorithm is named as hybrid *VNS* algorithm.
- The proposed algorithm can deal both the symmetric and asymmetric *TSPs* with equal efficiencies.
- At some instances the proposed algorithm shows better solution than what is available in the *TSPLIB* data set.
- The algorithm can solve various other problems in Statistics, Computer Science and Telecommunications.